

Practical no 1:

```
import pandas as pd
df = pd.read_csv("titanic.csv")
print(df.head())
print(df.isnull().sum())
print(df.describe())
print(df.info())
print(df.shape)
df['age'].fillna(df['age'].mean(), inplace=True)
df['sex'] = df['sex'].astype('category')
df['sex'] = df['sex'].cat.codes
print(df.head())
```

practical no 2:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'Name': ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'],
    'Math': [85, 90, np.nan, 40, 95, 100, 45, 200],
    'Science': [80, 85, 88, np.nan, 92, 98, 50, 110],
    'English': [78, 82, 85, 35, np.nan, 95, 40, 120]
})
print(df.isnull().sum())
df.fillna(df.mean(numeric_only=True), inplace=True)
Q1 = df[['Math', 'Science', 'English']].quantile(0.25)
Q3 = df[['Math', 'Science', 'English']].quantile(0.75)
IQR = Q3 - Q1
df[['Math', 'Science', 'English']] = np.where(
    df[['Math', 'Science', 'English']] > (Q3 + 1.5*IQR),
    Q3 + 1.5*IQR,
    df[['Math', 'Science', 'English']]
)
df[['Math', 'Science', 'English']] = np.where(
    df[['Math', 'Science', 'English']] < (Q1 - 1.5*IQR),
    Q1 - 1.5*IQR,
    df[['Math', 'Science', 'English']]
)
df['Math_log'] = np.log(df['Math'])
print(df)
```

practical no :3:

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset('titanic')
grouped = df.groupby('class')['fare'].agg(['mean', 'median', 'min', 'max', 'std'])
print(grouped)

fare_list = df.groupby('class')['fare'].apply(list)
print(fare_list)
```

practical no :4:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv("BostonHousing.csv")

X = df.drop('medv', axis=1)
y = df['medv']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
```

practical no 5:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

df = pd.read_csv("Social_Network_Ads.csv")

X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print(cm)

TN, FP, FN, TP = cm.ravel()

accuracy = (TP + TN) / (TP + TN + FP + FN)
error_rate = 1 - accuracy
precision = TP / (TP + FP)
recall = TP / (TP + FN)

print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

practical no 6:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix

df = pd.read_csv("Iris.csv")

X = df.drop('Species', axis=1)
y = df['Species']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print(cm)

accuracy = (cm.diagonal().sum()) / cm.sum()
error_rate = 1 - accuracy

print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
```

practical no 7:

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')

text = "Data analytics is the process of analyzing raw data to find patterns and insights."

tokens = word_tokenize(text)
print("Tokens:", tokens)

pos = pos_tag(tokens)
print("POS Tags:", pos)

stop_words = set(stopwords.words('english'))
filtered = [w for w in tokens if w.lower() not in stop_words]
print("After Stopword Removal:", filtered)

ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]
print("Stemming:", stemmed)
```

```
lm = WordNetLemmatizer()
lemmatized = [lm.lemmatize(w) for w in filtered]
print("Lemmatization:", lemmatized)
```

```
tfidf = TfidfVectorizer()
result = tfidf.fit_transform([text])
```

```
print("TF-IDF:", result.toarray())
```

```
practical no 8:
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = sns.load_dataset('titanic')
```

```
sns.histplot(df['fare'])
plt.show()
```

```
practical no 9
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = sns.load_dataset('titanic')
```

```
sns.boxplot(x='sex', y='age', hue='survived', data=df)
plt.show()
```

```
practical no 10:
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = sns.load_dataset('iris')
```

```
print(df.dtypes)
```

```
df.hist()
plt.show()
```

```
sns.boxplot(data=df)
plt.show()
```

```
print(df.describe())
```